Information Technology Course
Module Software Engineering
by Damir Dobric / Andreas Pech

ML 24/25-04 Implement the visualization of permanence value

Md Tiabur Rahman Tamim
tiabur.tamim@stud.fra-uas.de
Mat. Num-1428898

Md Shakaut kader
shakaut.kader@stud.fra-uas.de
Mat.Num-1427433

Zaria Fahamida
zaria.fahamida@stud.fra-uas.de
Mat.Num-1519657

Jakia Sultana Jui
jakia.jui@stud.fra-uas.de
Mat.Num-1430873

Abstract*— Hierarchical Temporal Memory (HTM) networks present a favorable computational framework for processing and analyzing high-dimensional data using Sparse Distributed Representations (SDRs). This experiment explores the reconstruction of encoded inputs within HTM algorithms, highlighting the accuracy and resilience of the reconstruction process. Our approach involves encoding two distinct types of input data numerical values and images and leveraging the HTM Spatial Pooler to generate SDRs. To reconstruct the original inputs from these encoded representations, we used the Reconstruct method provided by the Neocortex API. The performance of the reconstruction is assessed using similar measurements and visualization techniques, providing insight into the fidelity of the process. Our findings focus on the critical role of encoder selection in determining reconstruction accuracy, underscoring the need to employ diverse encoding strategies for different data types. In addition, we examine the challenges introduced by noise in encoded representations and its implications for the reconstruction process. Through detailed analysis and experimentation, our study contributes to the understanding and practical application of HTM networks in the domains of artificial intelligence and cognitive computing.

Keywords*— Hierarchical Temporal Memory (HTM), Reconstruction, Sparse Distributed Representations (SDRs), Encoders, Visualization Techniques.

I. Introduction

In the era of machine learning and neuroscience, Hierarchical Temporal Memory (HTM) marks an advance in our knowledge of how the mortal brain processes information. HTM, which was created by Numenta, goes beyond traditional machine literacy ways by fastening on the difficulties associated with temporal pattern recognition and using neocortical relief.

The neocortex, the focus of intellect in the mortal brain, serves as the model for HTM's framework. This creative fashion gives a fresh take on machine learning by combining the generalities of temporal scale and spareness, especially when evaluating successional data. More complex pattern recognition is made possible by its heavy emphasis on the commerce between spatial and temporal boundaries.

The Spatial Pooler (SP), the abecedarian medium for converting raw input into Sparse Distributed Representations (SDRs), is an essential part of HTM. This process is further than just a fine abstraction because it nearly resembles how the brain encodes information. Like the neocortex, these representations are sparse and distributed, which makes pattern discovery and memory storehouse more effective. The primary focus of the ongoing discussion into Hierarchical Temporal Memory (HTM) is the Reconstruct () system, a new addition to the Neocortex Api that serves as the Spatial Pooler's (SP) contrary operation. The objective of this project, titled" Visualization of Reconstructed Permanence Values," is to interpret the complex interactions between input and output within the HTM framework. Specifically, this research examines how the Reconstruct () method enables the Spatial Pooler to regenerate input sequences.

The high end of this study is to demonstrate the Spatial Pooler's capability to reconstruct input patterns. also, dissect two distinct representations of Permanence values one as a heatmap visualization of an image and the other as integer arrays. These representations correspond to two types of input data integer- grounded values and image- grounded inputs. The decoded sequences, structured as arrays of 0s and 1s, play a pivotal part in understanding the reconstructive capabilities of the Spatial Pooler.

In this paper, we conduct a detailed examination of the Reconstruct () function in the environment of HTM's Spatial Pooler. Beyond working with double numerical representations, we give an in depth analysis of the Reconstruct () process, deconstructing its functionality and illustrating Permanence values through both integer arrays and imaged heatmaps.

II. Literature Review

A. *Hierarchical Temporal Memory*

HTM incorporates factors comparable to the Spatial Pooler, which generates SDRs to efficiently execute high-

dimensional data. This frame has gained recognition across various disciplines, including finance, cybersecurity, robotics and health- care. Although HTM offers significant advantages, still there are some issues related to scalability and computational efficiency which stimulate continued research and development efforts.

Hawkins and Ahmad's foundational work [1] introduced the conception of sequence memory in the neocortex, pressing the part of Sparse Distributed Representations (SDRs) and prophetic coding mechanisms. These ideas form the base of Hierarchical Temporal Memory (HTM), a computational frame modelled after cortical processing.

The adding interest in HTM aligns with a broader shift toward biologically inspired artificial intelligence, potentially contributing to advancements in intellectual computing.

B. Sparse Distributed Representations (SDR's)

A fundamental element of HTM is the Spatial Pooler, which converts input data into SDRs, allowing the framework to process complex information aqueducts and acknowledge temporal patterns. The applicability of HTM extends across multiple disciplines, including cybersecurity, finance, healthcare, and robotics.

The research of Hawkins and Ahmad on the establishment of Sparse Distributed Representations (SDRs) [1] and analytical coding mechanisms placed the foundation for Hierarchical Temporal Memory (HTM), a computational framework inspired by cortical processing. Sparse Distributed Representations (SDRs) are characterized by their spareness and distributed nature, which serve as an efficient method for representing high-dimensional data within the HTM model.

Based on Hawkins and Ahmad's seminal work, George and Hawkins further developed the generalities, theoretical foundations and formal structure of HTM, providing a comprehensive perspective on its beginning principles. [2]

Despite its eventuality, HTM faces challenges related to scalability and computational effectiveness. However, ongoing study aim is to continue to upgrade and enhance the frame, emphasizing the significance of biologically inspired approaches in advancing cognitive computing.

C. Encoders

Encoders play an essential role in accelerating the metamorphosis of different data modalities, similar as multimedia inputs and timeseries data, into Sparse Distributed Representations (SDRs). This conversion enables HTM to effectively reuse and dissect complex datasets, perfecting its capability to do patterns and describe anomalies across various operations. Recent advancements in encoder design have concentrated on enhancing scalability, robustness to noisy or deficient data, and computational effectiveness. Rodriguez-Lujan et al. [3] introduced a new approach exercising self-supervised learning ways to strengthen encoder rigidity and adaptability within the HTM frame. Also, Smith et al. [4] explored the integration of deep learning methodologies for automated encoder discovery, thereby optimizing the design process and perfecting the encoder's capacity to accommodate a wide range of datasets and operation demands.

Encoder design is a continuously evolving field; however, remaining challenges impose ongoing research to enhance and optimize encoder algorithms and implementations within the HTM framework. These advancements play a crucial role in furthering cognitive computing and enabling HTM to address increasingly complex real-world applications.

D. Reconstruction in Terms of HTM:

In our study, a key focus was the development of a decoding strategy for Sparse Distributed Representations (SDRs) in specific contexts. Encoding scalar values using a 200-bit representation, it was necessary to gain a deeper understanding of existing methodologies. To achieve this, we examined the work of Mnatzaganian et al. [5], who outlined a systematic approach for determining activations based on specific permanence values. This process involves applying $\vec{c}$ as a masking function to $\vec{\phi}$, thereby obtaining the corresponding representation. Once gained, $\vec{\phi}$ is applied to compute the overall permanence associated with the attributes. This method produces valid probability values for each attribute; however, some probabilities may fall outside the expected range of 0,1. To rectify this, a dimensionality reduction technique is employed by thresholding the probability at ρ_s, as illustrated in Equation (1), where $\vec{u} \in \{\mathbf{0}, \mathbf{1}\}^{1\times n_a}$ represents the reconstructed input.

Furthermore, Mnatzaganian et al. [5] referenced the work of Thornton et al. (2013), who employed a similar methodology to analyze the interpretation of input data within the Spatial Pooler (SP). A significant advantage of this approach is its capability to reconstruct input solely based on permanence values and active columns, effectively decoupling the input from the system [6]. The equation governing this process is as follows:

$$\vec{\boldsymbol{U}} = I([max(\vec{\boldsymbol{c}}_i \vec{\boldsymbol{\phi}}_a \forall_i) \geq \rho_s])\forall_a \qquad (1)$$

Equation 1: Input Reconstruction Formula

In essence, Equation 1 calculates the reconstructed input vector ($\vec{u}$) by evaluating whether the maximum activation level for each attribute (a) determined by multiplying the learned representation values ($\vec{c}$) with the corresponding permanence values($\vec{\phi}$) surpasses the threshold (ρ_s). If the condition is satisfied for all attributes, the reconstructed input vector is assigned a value of 1, indicating activation; otherwise, it is assigned a value of 0. This approach enables the decoupling of the input from the HTM system, facilitating its reconstruction exclusively based on the learned representations within the Spatial Pooler.

III. METHODOLOGY

The goal of our methodology is to rebuild precisely the original input data. In the beginning, a variety of input data are presented, such as photos and numerical values between 0 and 99. An Image Binarizer is used to transform images into binary representations for image data. The input data is then transformed into integer arrays (int []), which are the only source of input for the experiment and are made up of 0s and 1s.

The encoded int [] arrays are then converted into Sparse Distributed Representations (SDRs) using the Hierarchical Temporal Memory (HTM) Spatial Pooler. The reconstruction process starts as soon as the SDRs are reached. The original encoded representations are methodically reconstructed using Neocortex Api's Reconstruct () method, which is based on the permanence values that the Reconstruction method returns.

Heatmaps and int [] sequences are the two main visualization types used in the methodology's last step. The accuracy of the reconstruction process is further evaluated by comparing the original input arrays with the reconstructed ones using a similarity analysis. The main objective is to assess how well the HTM algorithm reproduces inputs using Neocortex Api's Reconstruction technique, guaranteeing an accurate replication of the original encoded int [] structures. (Figure 1)

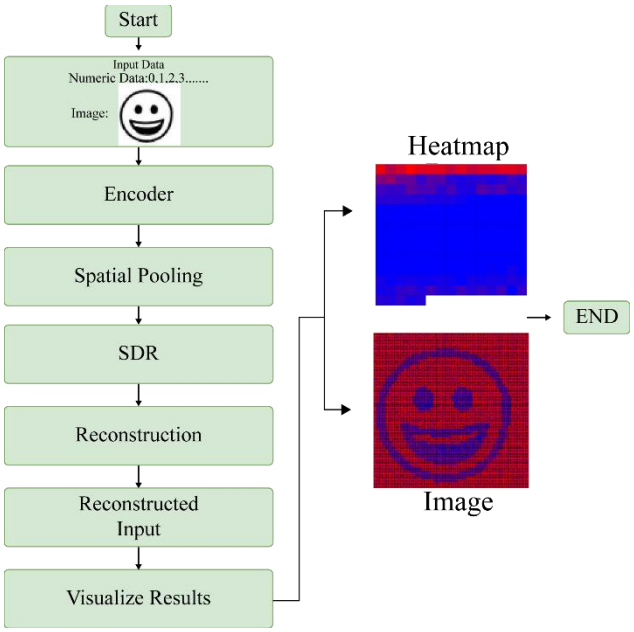

Figure 1: Graphical Representation of the Experiment

A. Input Types for the Experiment

1) Numerical Input Encoding: Numerical values ranging from 0 to 99 are processed using a scalar encoder, which encodes each value into a 200-bit representation. These encoded representations are then converted into integer arrays (int[]), with each array corresponding to a distinct numerical input (Figure 2).

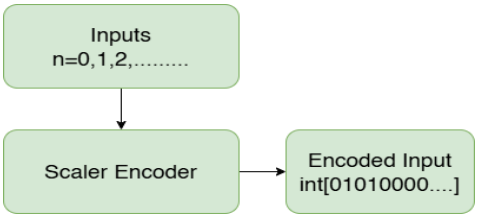

Figure 2: Encoded Inputs for Images

2) Image Input Encoding: An Image Binarizer is used to design binary representations for image data. The encoded arrays' confines match those of the source images. For instance, an encoded array of size 784 is created from a 28x28 pixel image, with each pixel represented by a binary value. (Figure 3).

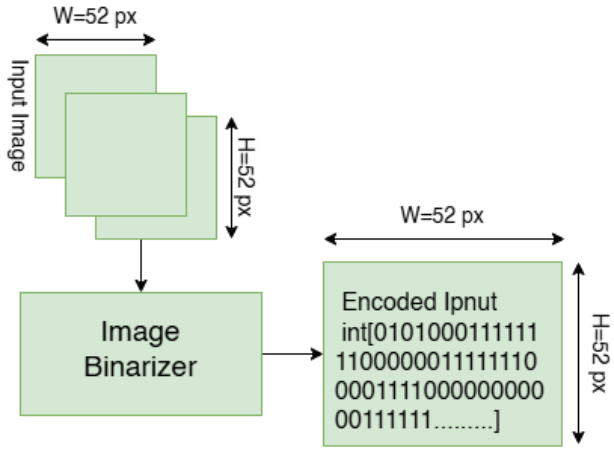

Figure 3: Encoded Inputs for Images

Our experiment uses both kinds of encoded arrays as input data and uses the Hierarchical Temporal Memory (HTM) architecture to precisely reconstruct the original input.

B. Reconstruction Method

Before commencing our project and exploring the details of Hierarchical Temporal Memory (HTM) and the Neocortex API, we established a strong foundational understanding by engaging with instructive HTM School videos available on YouTube. These succinct but insightful videos provided a thorough rundown of basic ideas, such as encoder mechanics, HTM settings, and the capabilities of the Spatial Pooler. This initial stage gave us crucial information that allowed us to approach our research with expertise and organization. The HTM School videos served as a valuable educational resource, increasing our comprehension of HTM principles and helping the effective utilization of the Neocortex API.

A critical component of the Neocortex API is the Reconstruct () method, designed to reverse the encoding process and reconstruct the original input from the Sparse Distributed Representations (SDRs) generated by the Spatial Pooler (Figure 4). A detailed explanation of this method's functionality is provided below:

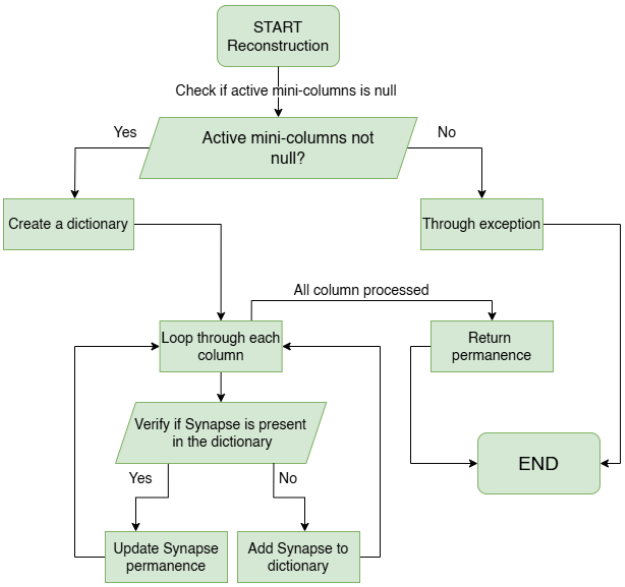

Figure 4: Graphical illustration of the workflow of the Reconstruction Method

1) ***Input Validation***: To guarantee the integrity of the input data, the procedure starts with thorough validation. An Argument Null Exception is raised if the input array of active mini columns is discovered to be null. On the other hand, the procedure moves forward with the reforming process if the input array is valid.

2) ***Column Retrieval***: The next step is extracting the set of columns from the current connections that correspond to the active mini-columns. Within the Reconstruct() system, the activeMiniColumns parameter is an integer array that specifies the indicators ofmini-columns linked as active. In Hierarchical Temporal Memory(HTM) and related models, a mini-column refers to a group of cortical neurons that activate in response to specific input patterns, similar as overlapping patterns or sensory inputs. The current approach utilizes activeMiniColumns to identify which mini-columns are active. Following this, a dictionary containing the cumulative permanence values of the synapses linked to these active mini-columns is reconstructed. These permanence values play a crucial role in determining the strength of neural

connections within the cortical region.

3) **Reconstruction Process:** The procedure progresses by repeating through each column and retrieving the synapses in its proximal dendrite.

- Synapses: In artificial neural networks, synapses stand for the connections between computational units or between neurons or dendrites in biological brains. In HTM, synapses connect the proximal dendrites of columns to either the input data or other columns in the network; their permanence value reflects the likelihood that they will help activate the related column in response to input. Since the permanence values of these synapses are modified in response to input patterns and network activity, they are indispensable to learning and adaptability in HTM. Each synapse is associated with a permanence value, which determines the strength of its connection.

- Proximal Dendrite: Each column in the cortical model of Hierarchical Temporal Memory (HTM) has a proximal dendrite to serve several vital functions. In the HTM network, proximal dendrites are connected to synapses, which stand for the connections between the input space and the columns. These dendrites may catch input directly from the input data or from nearby columns. As the main interface for each column, the proximal dendrite captures spatial patterns from the input and is crucial to the spatial pooling process, in which columns vie to represent spatial patterns in the input data. Each input index's persistence values are gathered in the reconstructed input dictionary as part of the reconstruction process. After that, the procedure inserts the input index as a new key-value pair after updating this dictionary and determining whether it already exists. The process ends by mapping each input index to its matching persistence value and returning the rebuilt input as a dictionary.

C. *Proposed Method to Visualize Permanence Values*

The objective of this segment is to utilize the Reconstruct() method provided by the Spatial Pooler (sp) to reconstruct the permanence values from the active columns identified during the learning phase of the experiment. The flow chart below demonstrates the implementation of our proposed approach for visualizing these permanence values. (Figure 5)

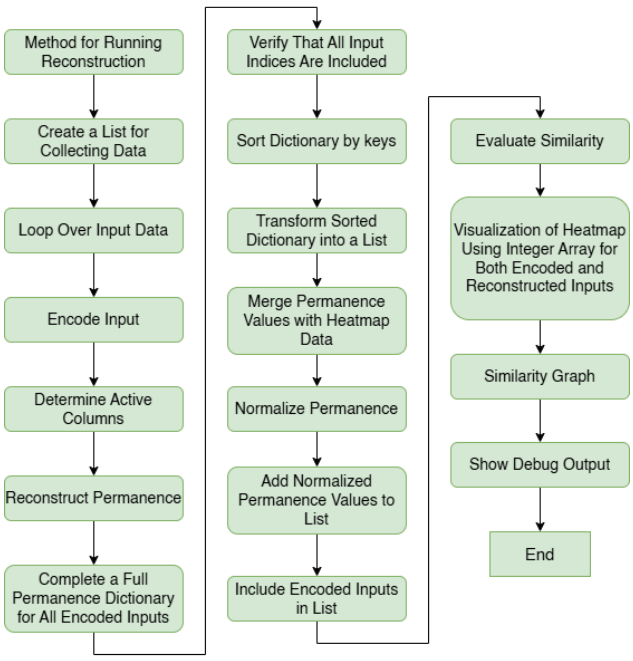

Figure 5: Graphical Representation of Running Reconstruction Method.

First, we encode each numerical value using the assigned encoder in our indicated method for inspecting persistence values and evaluating similarity for numerical inputs. For any

numerical input, this encoding procedure generates a Sparse Distributed Representation (SDR), which successfully captures its fundamental characteristics. The produced SDR is then used to identify which columns in the Spatial Pooler are active, making sure that learning is turned off to preserve the original input features.

Reconstructing the permanence values connected to the active features is the next crucial step, which is achieved using the Reconstruct () function that the Spatial Pooler. A dictionary called allPermanenceDictionary is created to fully account for all possible numerical input keys. To guarantee a detailed and correct representation of the numerical input space, this dictionary first adds the persistence values that match to the active columns.

The dictionary is systematically placed according to its keys in ascending order to guarantee reliability and make analysis easier. This ensures that the structure of all permanence dictionaries used. A subset of all possible input keys is represented by the reconstructed purpose values obtained from active columns; it should be noted. The persistence values corresponding to inactive columns are also included to provide a complete representation of all input bits and cover the whole spectrum of possible input indicators. The default persistence value for these inactive columns, which don't actively contribute to input representation, is 0.0. This meticulous approach ensures that the allPermanenceDictionary provides a complete and precise representation of all input indices, enabling a thorough analysis and visualization of the spatial patterns learned by the HTM network. A comprehensive study and display of the spatial patterns that the HTM network has learned are made possible by this methodical methodology, which guarantees that the allPermanenceDictionary offers a comprehensive and accurate representation of all input files.

Respectively, the reconstructed permanence values go through a normalization process based on a predefined threshold, enhancing comparison and enabling further analytical assessments. After normalizing, the permanence values are transformed into a list of numbers, streamlining subsequent processing steps. To assess the precision and consistency of the reconstruction process, Jaccard similarity is calculated between the original encoded inputs and the normalized permanence values, efficiently measuring the degree of similarity between the two representations.

Finally, leveraging both the heatmap data and the normalized permanence values, visually illuminating heatmaps are generated to provide understandings into the spatial patterns learnt by the HTM network from numerical inputs. Additionally, similarity plots are created to visually represent the level of similarity between the original encoded inputs and the reconstructed inputs, thereby offering a deeper understanding and validation of the reconstruction process.

Similarly, for image inputs, our proposed methodology follows a constructed approach designed to the individual properties of image data. The process begins with reading and binarizing each image file from the designated training folder, a fundamental step in preparing the images for spatial pooling. Once the binarization is complete, spatial pooling is applied, accelerating the detection of active columns that capture the most salient features within the images.

The subsequent step involves reconstructing the permanence values of the active columns, using the same approach used for numerical inputs. This ensures a comprehensive representation of the image space. By utilizing the allPermanenceDictionary, we systematically record all possible input indices, addressing any missing data points to achieve a holistic view of the input domain.

As with numerical data, the reconstructed permanence values undergo normalization and thresholding, followed by similarity computation to evaluate the accuracy of the reconstruction method. Ultimately, visualization techniques are applied to generate heatmaps, offering a graphical representation of the spatial patterns extracted from image inputs. In equivalent, similar plots are produced to illustrate the degree of correspondence between the original encoded inputs and their reconstructed counterparts, providing deeper insights and validation of the reconstruction process in the context of image data.

IV. IMPLEMENTATION OVERVIEW

As we begin our exploration within the Hierarchical Temporal Memory (HTM) framework, our primary focus is on visualizing and interpreting permanence values for both numerical and image-based inputs. This section of the paper delves into key aspects of our implementation, outlining our approach to developing a systematic method for adjusting and analyzing these values. By establishing thresholds, generating heatmaps, and evaluating similarities, we aim to reveal the intricate patterns and relationships encoded within the HTM network. Subsequently, the investigation delved into the Spatial Pooler algorithm. There was a consideration that perhaps input 51 to input 99 were not inhibiting the mini columns, prompting scrutiny of inhibition algorithms. The program employs a local inhibition algorithm for this purpose.

Our work is driven by the goal of gaining deeper insights into how HTM processes and organizes information over time, particularly across diverse data types. This understanding is essential for advancing our ability to interpret the network's internal mechanisms. In the following discussion, we will detail our methodology, starting with the adjustment of permanence values through thresholding probabilities and exploring their impact on the network's behavior.

A. Normalizing Permanence

To achieve consistency and comparability in the permanence values generated by the spatial pooler within the Hierarchical Temporal Memory (HTM) framework, we developed a thresholding function designed to normalize these values. This function is a critical component of our visualization and analysis pipeline, as it converts continuous permanence values into a more interpretable binary representation (see Figure 6).

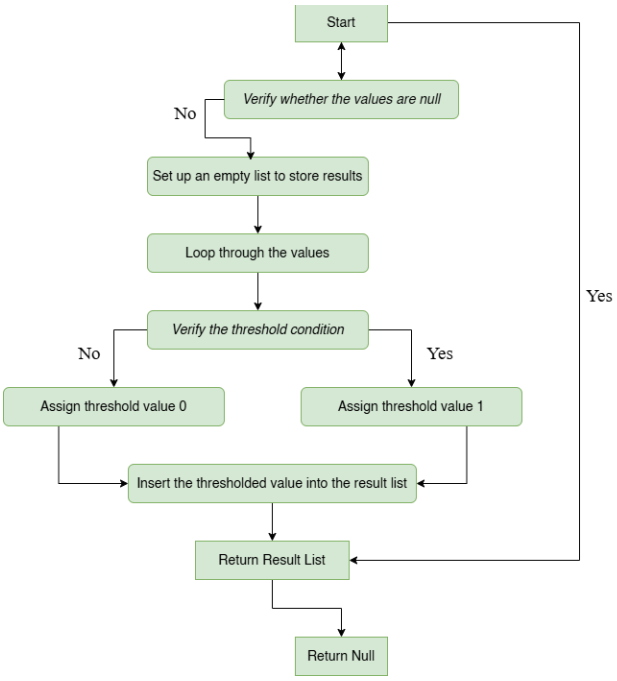

Figure 6: Graphical Representation of the Implemented Thresholding Probability Function

The selection of an appropriate threshold value is central to this normalization process. Through iterative experimentation and careful debugging, we identified optimal threshold values for both numerical and image inputs. For numerical data, the threshold was empirically set at 8.3, while for image inputs, it was determined to be 30.5. These values were chosen based on extensive testing and observation of the outputs, ensuring that the normalized permanence values accurately reflect the characteristics of the original encoded inputs. By systematically adjusting the threshold values and comparing the resulting outputs with the original data, we refined the normalization process to strike a balance between preserving the integrity of the input data and enabling meaningful analysis. This careful approach highlights the significance of parameter selection in effectively visualizing and interpreting permanence values within the HTM framework. Ultimately, this work enhances our understanding of hierarchical temporal processing across both numerical and image data domains.

B. Visualization with Heatmap

When we are working with heatmaps, it's important to understand how the colors represent different permanence values for each input index. The idea of "heat" in the heatmap shows how strong the connection is between the input data and the columns in the Hierarchical Temporal Memory (HTM) network. Warmer colors mean higher permanence values, which suggest a stronger chance that the synapse will help activate the connected column when it receives input. On the other hand, cooler colors indicate lower permanence values, meaning the synapse has less influence on activating the column. In simple terms, the color gradient in the heatmap gives us a visual way to see how the network interprets the input data, with warmer colors showing stronger connections and cooler colors showing weaker or no connections.

To fully grasp this, it's helpful to know how synapses and permanence values work in the HTM framework. Synapses connect the proximal dendrites of columns to the input data or other columns in the network. Each synapse has a permanence value that measures how strong the connection is. This value tells us how likely it is that the synapse will help activate the connected column when it gets input. By looking at the heatmap and understanding the color gradient, we can see how these synapses are weighted and how they affect the network's processing of both numerical and image inputs.

The process of generating a heatmap involves several steps that ensure the final visualization is accurate and meaningful. (Figure 7) It begins with determining the appropriate height and maximum length for the heatmap based on the input data. These dimensions are then verified to ensure they fit the data correctly. A bitmap is generated to serve as the base for the heatmap, and the background is removed to focus on the essential data. A title is added to provide context, and scale factors are calculated to ensure the heatmap accurately represents the data.

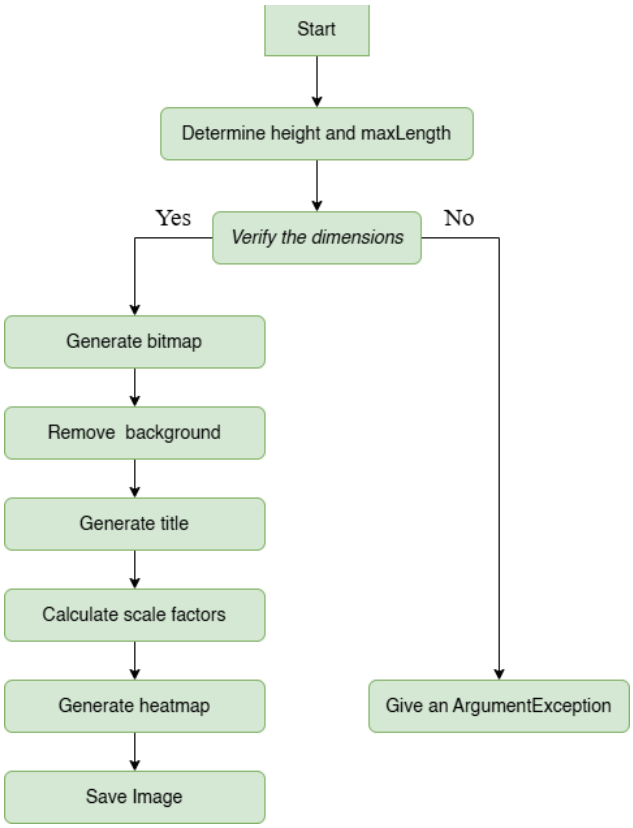

Figure 7: Graphical representation of Draw 1D Heatmap Function

Once these preparations are complete, the heatmap itself is generated by applying the color gradient based on the permanence values. This step is crucial as it visually represents the strength of connections within the network. During this process, any exceptions or errors are handled to ensure the integrity of the visualization. Finally, the

completed heatmap image is saved for further analysis or presentation.

The **Draw1dHeatmap** function plays a key role in this process, as it integrates heatmap data with normalized permanence values and the original encoded inputs. This integration allows us to create detailed visual representations that accurately depict the network's learning process. Each heatmap is carefully designed to highlight the relationship between input indices and their permanence values, providing a clear picture of how the HTM network interprets and processes the input data.

Additionally, by including the original encoded inputs alongside the heatmap data, we can better understand how the input data is translated and represented in the HTM network. This comparison helps us evaluate the accuracy of the network's representation and identify areas where improvements might be needed. The figure serves as a valuable tool for analyzing the network's performance and refining its interpretation of input data.

C. Similarity Calculation

In our experiment, we utilized the Jaccard similarity coefficient to validate the reconstructed output, which includes both numerical and image inputs. The Jaccard coefficient, also known as the Tanimoto coefficient, measures similarity by comparing the intersection of two sets to their union [7]. This metric is particularly well-suited for binary data, making it ideal for comparing the original encoded inputs and the reconstructed inputs in our study.

Mathematically, the Jaccard similarity coefficient J (A, B) between two sets A and B is defined as (Equation 2):

$$J(A,B) = \frac{A \cap B}{A \cup B} \qquad (2)$$

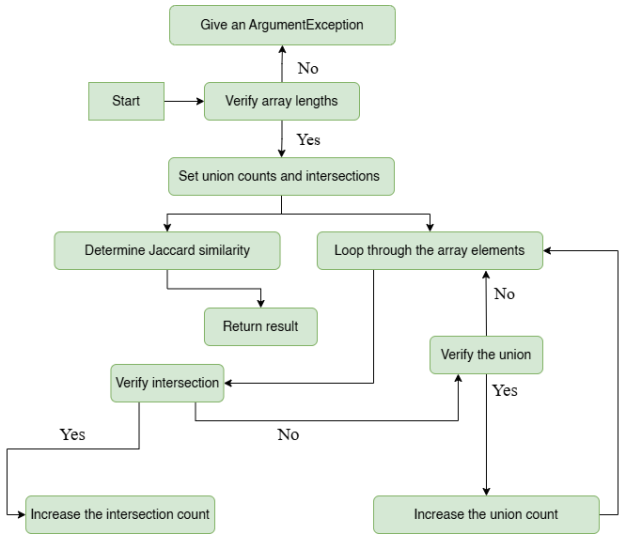

Figure 8: Graphical representation of the Implemented Jaccard similarity coefficient calculation function.

For numerical data, the method takes two integer arrays as input, representing binary values. It first verifies that the arrays have the same length; if not, it throws an exception. Then, it iterates through the arrays to count the number of intersecting elements (where both arrays have a value of 1 at the same index) and the total number of unique elements (where either array has a value of 1 at the index). Finally, it computes the Jaccard similarity coefficient by dividing the intersection count by the union count, ensuring a normalized value between 0 and 1.

For image data, the process is adapted to handle 2D matrices. The images are first converted into binary format using a thresholding technique. The method then verifies that the images have the same dimensions. It iterates through the pixel matrices to count intersecting pixels (where both images have a value of 1 at the same pixel location) and unique pixels (where either image has a value of 1 at the location). The Jaccard similarity coefficient is computed in the same manner as for numerical data.

This unified approach allows us to apply the Jaccard similarity coefficient consistently across both numerical and image inputs, enabling a robust validation of the reconstructed outputs. (See Figure 8 for a visual representation of this process.)

D. Similarity Graph Visualization

Visualizing similarity calculation graphs provides valuable insights into algorithm performance and the relationships between different data points. By plotting similarity scores across a range of inputs, researchers and practitioners can identify patterns, trends, and anomalies in the data. This visualization is instrumental in evaluating and validating similarity metrics, offering a clear understanding of how well the metrics capture meaningful relationships.

The similarity calculation graph offers a comprehensive overview of the similarities and differences between pairs of data points within a dataset. It helps researchers identify clusters of similar data points, detect outliers, and highlight regions of high or low similarity. These insights are crucial for understanding the underlying structure of the data and assessing the effectiveness of similarity measures. (See Figure 9 for an example.)

The "DrawCombinedSimilarityPlot" function plays a key role in visualizing the similarity between encoded inputs and reconstructed inputs. This function generates a graphical representation of the similarity values for each input, providing a clear understanding of the network's performance in reconstructing data.

The function creates a bitmap image with bars representing the similarity values. Each bar corresponds to an input, and its height reflects the similarity between the original encoded input and the reconstructed input. The function takes several parameters, including the list of similarity values, the file path for saving the image, and the dimensions of the image.

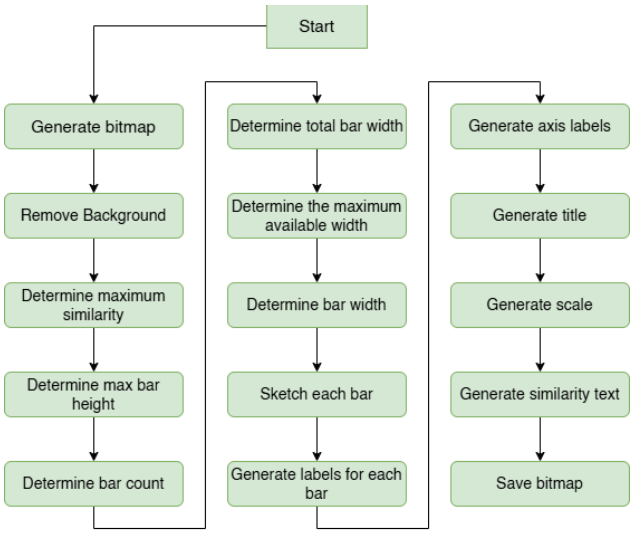

Figure 9: Graphical Representation of Implemented Draw Similarity Plot Function

Internally, the function calculates the maximum similarity value from the provided list to scale the bar heights appropriately. It then determines the width and spacing for each bar based on the specified dimensions. Next, it iterates through the similarity values, drawing a bar for each input. The height of each bar is proportional to its similarity value, offering a visual representation of how closely the reconstructed input matches the original encoded input.

Additionally, the function adds axis labels, a title, and a scale to the image to provide context and enhance interpretability. This ensures that the generated image is both informative and visually appealing, enabling users to quickly assess the performance of the HTM network in reconstructing input data.

V. RESULT ANALYSIS

In this project, we implemented a Spatial Pattern Learning experiment using the NeoCortexApi, a framework inspired by Hierarchical Temporal Memory (HTM), which is designed to mimic the learning mechanisms of the human neocortex. The primary objective of this study was to investigate the reconstruction of input patterns in HTM networks using the NeoCortex API's Reconstruct method. Specifically, we aimed to explore the reconstruction process, visualize permanence values, and calculate the similarity between original encoded inputs and reconstructed outputs. To achieve this, we conducted experiments on two types of data: numerical data and image data. Here, we focus on the implementation and results for numerical data, which serves as a foundational demonstration of the reconstruction process. For numerical data, we used scalar values ranging from 0 to 99 as input. These values were encoded into Sparse Distributed Representations (SDRs) using a Scalar Encoder, which transforms scalar inputs into binary vectors with a small percentage of active bits. The encoded data was then fed into the Spatial Pooler (SP), a core component of HTM, and trained over multiple cycles to learn spatial patterns. To ensure stable learning, we employed the Homeostatic Plasticity Controller (HPC), which monitored the SP's stability and triggered events when the system reached a stable state. Once trained, we reconstructed permanence values, representing the strength of synaptic connections, and normalized them to binary values (0 or 1) using a predefined threshold value of 8.3, which was chosen because it provided the best results for reconstructing the input patterns. This allowed us to compare the original encoded inputs with the SP's reconstructed outputs. To visualize the results, we generated two key figures: Figure 10 and Figure 11 which are given below.

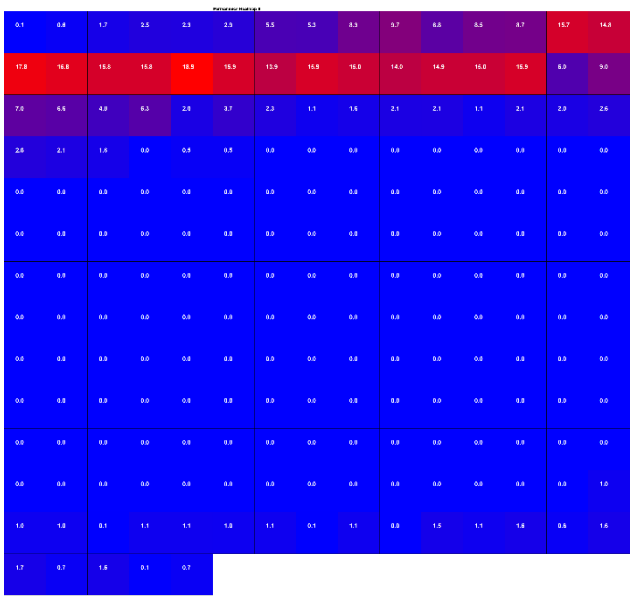

Figure 10: Reconstructed Heatmap Image for Numerical Input

Figure 10 Reconstructed Heatmap Image for Numerical Input illustrates the evolution of permanence values over time, providing a detailed view of how the SP internalizes and represents numerical input patterns. The heatmap reveals the progression of permanence values across different input patterns, demonstrating how the SP gradually stabilizes and accurately reconstructs the input data. Each row in the heatmap corresponds to a specific input value, while the columns represent the permanence values of the SP's columns. The color intensity in the heatmap reflects the strength of permanence values, with darker shades indicating higher permanence. This visualization highlights the SP's ability to learn and adapt to spatial patterns, showcasing the dynamic nature of synaptic plasticity during the learning process. The heatmap also reveals areas where the SP successfully captures the input patterns, as well as regions with minor discrepancies, providing insights into the SP's learning efficiency.

Figure 11 Encoded vs. Reconstructed Comparison (After Thresholding) provides a direct comparison between the original encoded inputs and the SP's reconstructed outputs. In this heatmap, green cells indicate a perfect match between the encoded and reconstructed values (i.e., both are 1), while red cells indicate mismatches (i.e., one is 0 and the other is 1). This visualization offers a clear and intuitive representation of the SP's accuracy in reconstructing input patterns. For example, the heatmap shows that the SP was able to accurately reconstruct most of the numerical patterns, with

green cells dominating the visualization. However, there are some red cells in certain regions, indicating minor discrepancies where the SP failed to perfectly match the original input. These mismatches are primarily due to the SP's learning process, which involves balancing stability and plasticity. Overall, Figure 11 demonstrates the SP's effectiveness in learning and reconstructing spatial patterns, with high accuracy and only minor errors

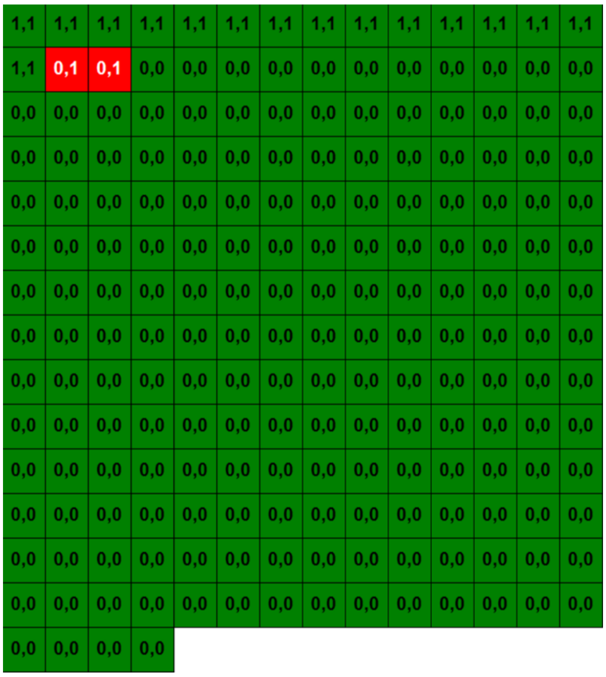

Figure 11: Encoded vs. Reconstructed Comparison (After Thresholding)

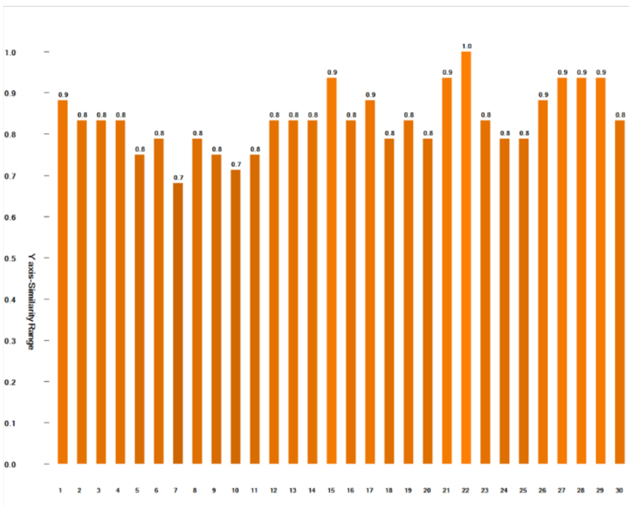

Figure 12: Similarity Graph for the Numerical Input.

To further quantify the accuracy of the SP, we calculated the similarity between the original encoded inputs and the reconstructed outputs using the Jaccard Similarity Index. The similarity values for each input were stored and visualized in Figure 12 Similarity Plot, which shows the distribution of similarity scores across all input patterns. The plot reveals that the majority of inputs have high similarity scores, indicating that the SP successfully reconstructed most of the input patterns with high accuracy.

Following the successful implementation and analysis of the numerical data experiment, we extended our Spatial Pattern Learning study to handle image data, using the Neocortex Api framework inspired by Hierarchical Temporal Memory (HTM). The objective of this extension was to investigate how the Spatial Pooler (SP) reconstructs image patterns and to visualize the learning process for more complex, high-dimensional data. For this experiment, we used a set of grayscale images, which were resized to a uniform dimension of 52x52 pixels. Each image was binarized, converting pixel values into binary vectors (0s and 1s), and then encoded into Sparse Distributed Representations (SDRs). These SDRs were processed by the SP, which learned to recognize spatial patterns within the images. The SP was trained over multiple cycles, with the Homeostatic Plasticity Controller (HPC) ensuring stability during the learning process. Once trained, we reconstructed permanence values, representing the strength of synaptic connections, and normalized them to binary values (0 or 1) using a predefined threshold value of 67.0, which was chosen because it provided the best results for reconstructing the image patterns. This allowed us to compare the original binarized images with the SP's reconstructed outputs. To visualize the results, we generated three key figures: Figure 13, Figure 14, and Figure 15.

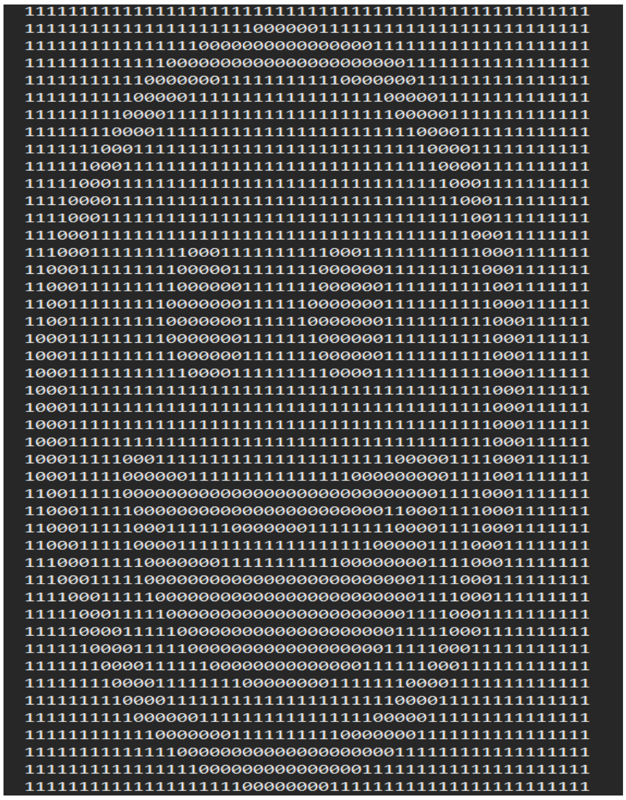

Figure 13: Binary Image as Text Encoded

Figure 14 illustrates the evolution of permanence values over time, providing a detailed view of how the SP

internalizes and represents image patterns. The heatmap reveals the progression of permanence values across different image inputs, demonstrating how the SP gradually stabilizes and accurately reconstructs the image data. Each row in the heatmap corresponds to a specific image, while the columns represent the permanence values of the SP's columns. The color intensity in the heatmap reflects the strength of the permanence values, with darker shades indicating higher permanence. This visualization highlights the SP's ability to learn and adapt to spatial patterns in images, showcasing the dynamic nature of synaptic plasticity during the learning process. The heatmap also reveals areas where the SP successfully captures the image patterns, as well as regions with minor discrepancies, providing insights into the SP's learning efficiency.

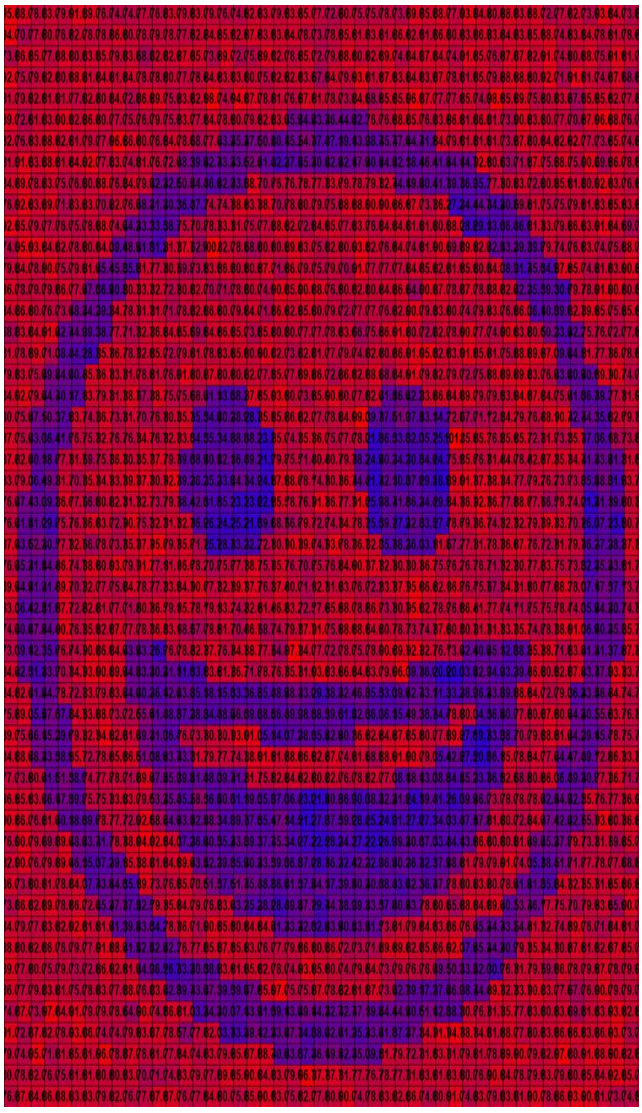

Figure 14: Image Heatmap

Figure 15 which is given below provides a direct comparison between the original binarized images and the SP's reconstructed outputs. In this figure, the reconstructed binary image is displayed, showing how closely the SP's output matches the original input. The figure demonstrates the SP's ability to accurately reconstruct most of the image patterns, with only minor discrepancies in certain regions. These discrepancies are primarily due to SP's learning process, which involves balancing stability and plasticity. Overall, Figure 15 demonstrates the SP's effectiveness in learning and reconstructing image patterns, with high accuracy and only minor errors.

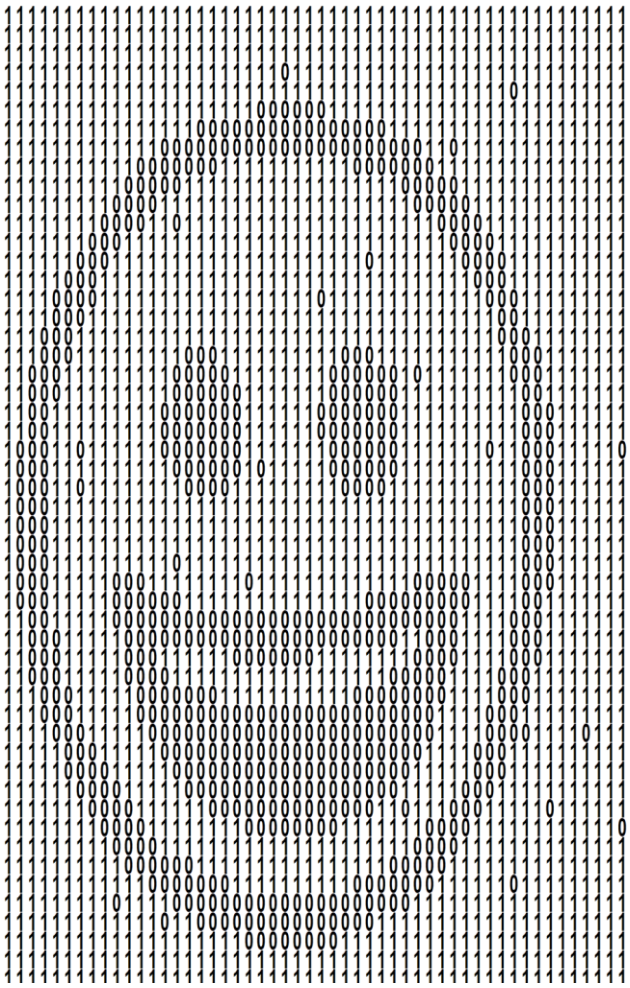

Figure 15: Binary Image reconstructed

To further quantify the accuracy of the SP, we calculated the similarity between the original binarized images and the reconstructed outputs using the Jaccard Similarity Index. The similarity values for each image were stored and visualized in Figure16.

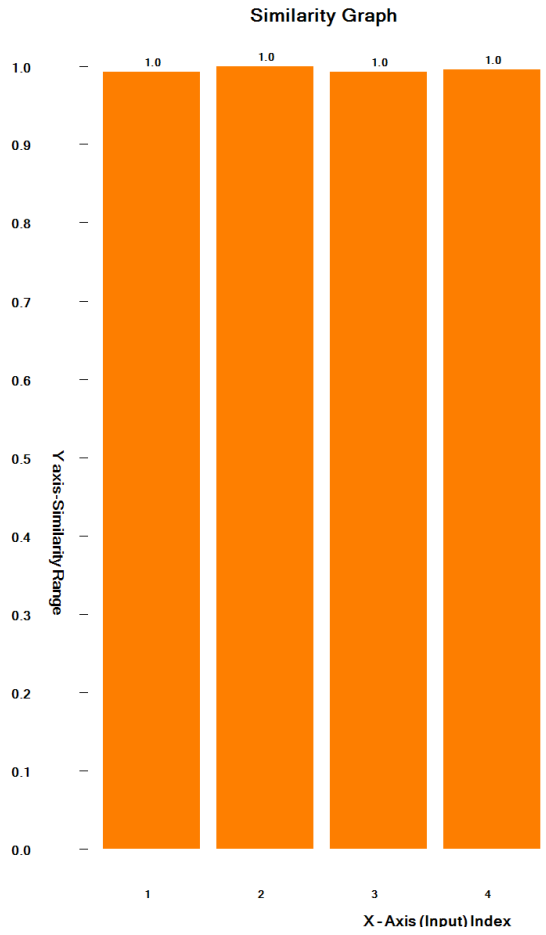

Figure 16: Similarity Graph for the Image Inputs 1-4

These plots show the distribution of similarity scores across all image inputs, revealing that the majority of images have high similarity scores. This indicates that the SP successfully reconstructed most of the image patterns with high accuracy. While the code does not explicitly calculate the average similarity score, the distribution of similarity values in Figure 16 suggests that the SP achieves a high level of accuracy in reconstructing image patterns.

VI. DISCUSSION

The discussion section provides a thorough analysis of the study's findings, their implications, and the broader context in the field of Hierarchical Temporal Memory (HTM), highlighting key areas for further exploration. A major challenge lies in the presence of noise within the encoded Sparse Distributed Representations (SDRs), which can distort underlying patterns and compromise reconstruction accuracy. While our current study did not explicitly explore this scenario, it underscores a critical area for future investigation. Enhancing the noise robustness of decoding algorithms could significantly improve the reliability and practical utility of HTM networks, particularly in real-world applications where data integrity is often compromised. Additionally, our observation regarding the impact of encoder choice on HTM network performance highlights an essential consideration in network design and optimization. The discrepancy in performance between using different encoders such as the Scalar Encoder for numerical inputs and the Image Binarizer for image data underscores the significance of encoder selection in achieving optimal outcomes. Further exploration into the effects of diverse encoding strategies on reconstruction accuracy and robustness could yield valuable insights for tailoring HTM networks to accommodate a wide range of input data types and enhance their effectiveness in various applications. Moreover, the use of predefined threshold values for normalizing permanence values played a crucial role in the reconstruction process, with thresholds of 8.3 for numerical data and 67.0 for image data providing the best results. Future research could explore adaptive thresholding techniques to dynamically adjust thresholds based on input data, potentially improving reconstruction accuracy. By addressing these challenges and exploring new directions, we can further enhance the capabilities of HTM networks and unlock their full potential in the field of machine learning and artificial intelligence.

VII. CONCLUSION

To sum up, our research has yielded significant knowledge about how to reconstruct Sparse Distributed Representations (SDRs) in Hierarchical Temporal Memory (HTM) networks. We have investigated the complexities of the HTM framework and its use in reconstructing encoded inputs by implementing and analyzing various components, including input encoding, reconstruction algorithms, similarity calculations, and visualization techniques. The significance of encoder selection in affecting reconstruction accuracy and network performance is one of our study's main conclusions. Different encoding strategies, such as the Scalar Encoder for numerical data and the Image Binarizer for image data, produced varying results, emphasizing the importance of carefully selecting encoding techniques based on the type of input. Furthermore, our analysis of similarity computation techniques, including the Jaccard Similarity Index, provided a quantitative measure of reconstruction fidelity, enabling a precise assessment of reconstruction accuracy and network efficiency. Overall, our work lays the groundwork for further study and advancement in this area by improving our understanding of and ability to use HTM networks for reconstructing encoded inputs. HTM networks have the potential to provide robust solutions for a range of data processing and analysis tasks through further research and development, advancing the fields of cognitive computing and artificial intelligence.

VIII. REFERENCES

[1] J. &. A. S. (. Hawkins, "Why neurons have thousands of synapses, a theory of sequence memory in neocortex," Frontiers, pp. 10,23, 30 March 2016.

[2] D. &. H. J. George, "A Hierarchical Temporal Memory Model for Learning and Recognition of Sequential Data.," in

IEEE Transactions on Pattern Analysis and Machine Intelligence, 2011.

[3] I. G.C. L. &. S. A. Rodriguez-Lujan, "Enhancing Encoder Robustness in Hierarchical Temporal Memory with Self-Supervised Learning. Conference/Journal," January 2020.

[4] A. J. B. &. P. R. Smith, "Automated Encoder Discovery in Hierarchical Temporal Memory using Neural Architecture Search. Conference/Journal," January 2020.

[5] J. F. E. &. K. D. Mnatzaganian, " A mathematical formalization of hierarchical temporal memory's spatial pooler. Frontiers in Robotics and AI, 3," vol. DOI: 10.3389/frobt.2016.00081, January 2017.

[6] J. &. S. A. Thornton, "Spatial pooling for greyscale images. International Journal of Machine Learning and Cybernetics, 4(3), 207–216," Vols. DOI: 10.1007/s13042-012-0087-7, 2012.

[7] G. K. a. V. K. M. Steinbach, Similarity Measures for Text Document Clustering," in Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2000, pp. 48 57.